Energy Dissipative Deep Operator Neural Networks for solving Partial Differential Equations

A B. S. Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science

by

Shantanu Vishal Punde
(2101MC53)

under the guidance of

Dr. Kuldip Singh Patel

to the

DEPARTMENT OF MATHEMATICS

INDIAN INSTITUTE OF TECHNOLOGY PATNA
PATNA - 801106, BIHAR

DECLARATION BY THE STUDENT

I certify that:

- The work contained in this thesis is original and has been done by me under the guidance of my supervisor.
- The work has not been submitted to any other Institute for any degree or diploma.
- I have followed the guidelines provided by the Institute in preparing the thesis.
- I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- Whenever I have used materials (data, theory and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the reference section.
- The thesis has been checked by anti-plagiarism software.

Shantanu Vishal Punde

CERTIFICATE

*This is to certify that the work contained in this thesis entitled "**Energy Dissipative Deep Operator Neural Networks for solving Partial Differential Equations**" is a bonafide work of **Shantanu Vishal Punde** (**Roll No. 2101MC53**), carried out in the Department of Mathematics, Indian Institute of Technology Patna under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Dr. Kuldip Singh Patel**

Assistant Professor,

May, 2025 Department of Mathematics,

Patna. Indian Institute of Technology Patna, Bihar.

Acknowledgements

I want to extend my heartfelt appreciation to my thesis advisor, Dr. Kuldip Singh Patel, for his indispensable guidance, ongoing support, and constructive feedback throughout this research journey. His mentorship has played a crucial role in the development of both this thesis and my academic progression.

I would also like to express my gratitude to Mr. Keshav Saxena, my fellow colleague and classmate, for his insightful contributions and assistance along the way.

Finally, I am incredibly grateful to my family for their steadfast love, patience, and support throughout my undergraduate education. Their faith in me helped me persevere during the most challenging moments.

Shantanu Vishal Punde

Date: 4th May 2025

Place: Indian Institute of Technology Patna

Contents

1 Introduction **1**

1.1 Operator Learning and Neural Networks 1

1.2 Deep Operator Networks for PDE Solutions 2

1.3 Methodology 3

1.4 Weight Update Algorithm 6

1.5 Loss Function 7

1.6 Discussion 7

2 Review of Prior Works **9**

2.1 Recent Works on DeepONet 9

2.2 Methods to Solve Partial Differential Equations 10

2.2.1 Separation of Variables 10

2.2.2 Wavelet-Optimized Compact Finite Difference (WOCFD) Method . 11

2.2.3 Finite Difference Method (FDM) 12

2.3 Motivation 13

3 Numerical Illustrations **15**

3.1 Heat Equation 15

3.1.1 PDE, Boundary Conditions, Spatial and Temporal Domain 15

3.1.2 Data Generation and Model Training 16

3.1.3 Plots and Error Matrix 17

3.2 Parametric Heat Equation 18
3.2.1 PDE, Boundary Conditions, Spatial and Temporal Domain 18
3.2.2 Data Generation and Model Training 19
3.2.3 Plots and Error Matrix 20
3.3 2D Convection Diffusion Equation 21
3.3.1 PDE, Boundary Conditions, Spatial and Temporal Domain 22
3.3.2 Data Generation and Model Training 22
3.3.3 Plots and Error Matrix 23
3.4 Analysis of Illustrations 26

4 Conclusion & Discussion **29**
4.1 Conclusion 30
4.2 Future Work 30

Chapter 1

Introduction

Learning operators is a fundamental challenge with significant consequences in both science and engineering. The process of estimating operators in Euclidean and Banach spaces introduces novel design challenges for neural networks, while also creating possibilities for tackling intricate systems. In relation to partial differential equations (PDEs), operator learning facilitates accurate solution predictions by leveraging the deep prior knowledge integrated into neural network structures.

1.1 Operator Learning and Neural Networks

The universal approximation theorem illustrates that neural networks have the ability to approximate any continuous function if they possess an adequate number of layers and neurons in their hidden layers. An often-overlooked extension indicates that even architectures with a single hidden layer can approximate nonlinear continuous functionals (which map function spaces to scalars) and operators (which map between infinite-dimensional function spaces), provided certain conditions related to the activation functions are met. This ability establishes the theoretical basis for operator learning frameworks such as DeepONets, which focus on addressing partial differential equations by approximating solution

operators.

Theorem: 1 (Universal Approximation Theorem for Nonlinear Operators) *Let $\Omega_1 \subset X$ be a compact subset of a Banach space X, V a compact set in $C(\Omega_1)$, and $\Omega_2 \subset R^d$ a compact subset. Consider a continuous nonlinear operator $G : V \to C(\Omega_2)$ and a non-polynomial continuous activation function σ. For any prescribed accuracy $\epsilon > 0$, there exist integers M, N, m, real-valued constants $c_k^i, \zeta_k, \xi_{ij}^k, \theta_i^k$, spatial points $\omega_k \in R^d$, and sensor locations $\{x_j\}_{j=1}^m \subset \Omega_1$ such that the following approximation holds uniformly:*

$$\left| G(u)(y) - \sum_{k=1}^{N} \sum_{i=1}^{M} c_k^i \, \sigma \left(\sum_{j=1}^{m} \xi_{ij}^k u(x_j) + \theta_i^k \right) \sigma(\omega_k \cdot y + \zeta_k) \right| < \epsilon$$

for all input functions $u \in V$ and evaluation points $y \in \Omega_2$.

1.2 Deep Operator Networks for PDE Solutions

Operator learning neural networks are designed to approximate operators instead of individual functions, specifically focusing on those that characterize solutions to partial differential equations (PDEs). Given an operator $G : \Omega^* \to \Omega^*$, the objective of the network is to forecast $G(u)(y) \in R$ for every evaluation point $y \in \Omega$, and it is trained on discrete representations of the operator.

In practice, the input functions u are discretized through sensor points $\{x_1, \ldots, x_m\}$, and the inputs to the network are arranged as $[u(x_1), \ldots, u(x_m)]$ along with y. The loss function evaluates the discrepancy between the predicted output q and the actual operator output $G(u)(y)$. For PDEs that depend on time, the training data is formatted as triplets $(u, y, G(u)(y))$, where u is discretely represented at the sensor points.

1.3 Methodology

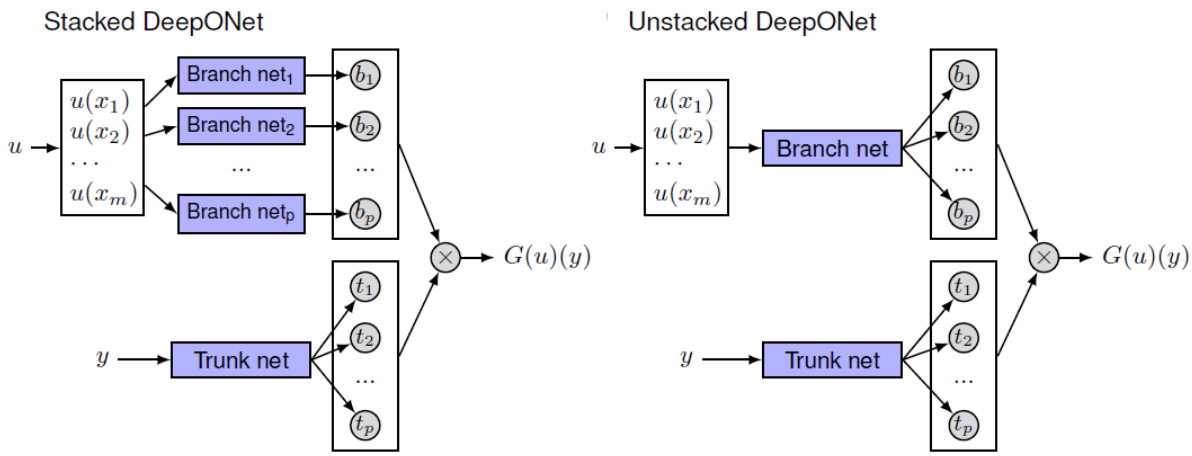

Fig. 1.1 DeepONet Architecture

There are two potential configurations of DeepONet: Stacked DeepONet and Unstacked DeepONet. We focus on learning operators within a larger framework, with the primary emphasis on the consistency of the sensors

$$\{x_1, x_2, \ldots, x_m\}$$

for input functions. Within this broad framework, the network inputs consist of two separate components:

$$\big[u(x_1),\, u(x_2),\, \ldots,\, u(x_m)\big]^T \quad \text{and} \quad y,$$

to ensure robust performance while considering the architecture's design.

A fundamental method is to employ a conventional network (such as FNN, CNN, or RNN) and combine the two input elements as the input to the network,

$$\big[u(x_1),\, u(x_2),\, \ldots,\, u(x_m),\, y\big]^T.$$

Nonetheless, this lacks a specific structure, making it less effective to choose networks like

CNN or RNN. In our scenario, the reference model is a fully-connected feedforward network (FNN).

In situations involving high dimensions, $y \in R^d$ represents a vector with d elements, meaning the dimension of y does not align with the dimension of each scalar $u(x_i)$ for $i = 1, 2, \ldots, m$. Consequently, $u(x_i)$ and y cannot be processed in the same way; thus, we require at least two subnetworks to manage $\big[u(x_1),\, u(x_2),\, \ldots,\, u(x_m)\big]^T$ and y individually. These two subnetworks are referred to as the Branch Net and the Trunk Net.

Branch Net. This network processes the input function u at various sensor locations $\{x_i \in \Omega \mid i = 1, \ldots, m\}$. The output layer features p neurons, each corresponding to a scalar value

$$b_j \;=\; b_j\big(u(x_1),\, u(x_2),\, \ldots,\, u(x_m)\big), \quad j = 1, 2, \ldots, p.$$

Trunk Net. This network encodes a range of evaluation points $\{y_k \in \Omega \mid k = 1, \ldots, n\}$. Similarly, its output layer contains p neurons, each generating

$$g_j \;=\; g_j\big(y_1,\, y_2,\, \ldots,\, y_n\big), \quad j = 1, 2, \ldots, p.$$

The evaluation points $\{y_k\}$ can be selected freely to compute the loss function.

Both the Branch Net and the Trunk Net share the same number of neurons in their final layers, allowing the output of the DeepONet to be represented as the inner product of the two p-dimensional vectors:

$$\mathcal{G}(u)(y) \;=\; \sum_{j=1}^{p} b_j\big(u(x_1), \ldots, u(x_m)\big)\, g_j\big(y_1, \ldots, y_n\big).$$

This setup forms a clear link between the expected output and the output produced.

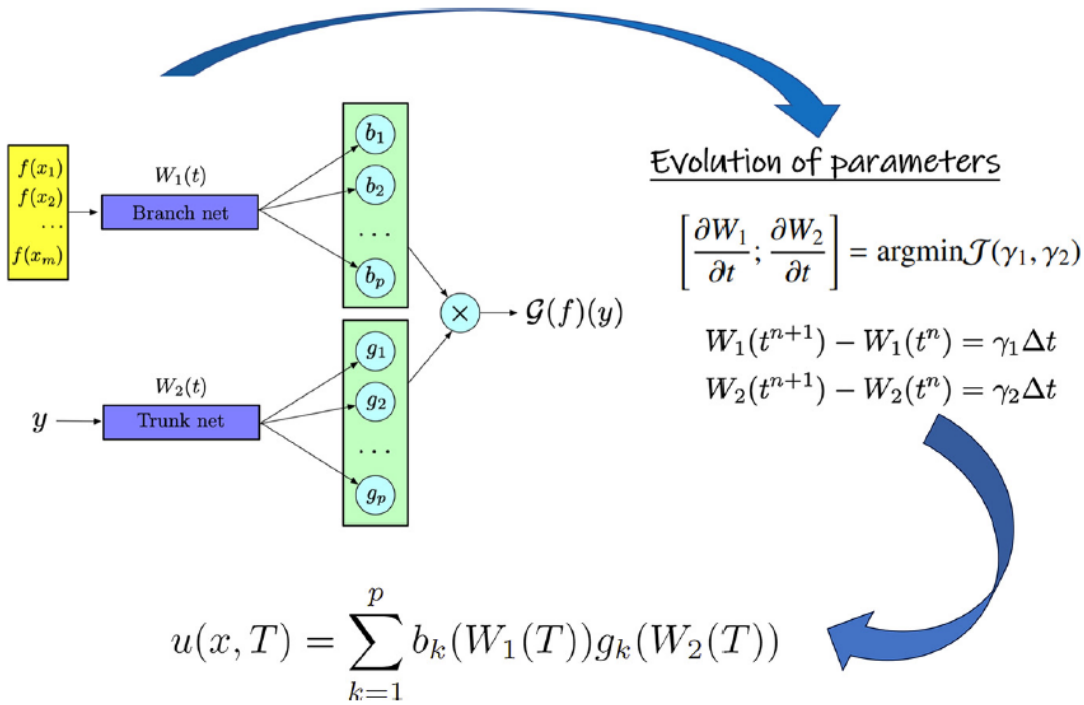

Fig. 1.2 Evolution of parameters in unstacked DeepONet Architecture

The phrase "Energy-Dissipative" signifies the network's integration of a fundamental principle from gradient flow systems: energy must decrease consistently over time. This requirement is maintained through the constraint $(r^{n+1})^2 \leq (r^n)^2$, which ensures that energy is dissipated at each time step. Such a characteristic is crucial for making physically reliable long-term predictions in systems such as the Allen-Cahn equation, allowing for accurate forecasts based solely on initial data.

DeepONets accomplish this through a specialized training method that combines back-propagation with physics-informed optimization techniques. The loss function inherently incorporates energy dissipation laws, thereby ensuring thermodynamic consistency while approximating operators.

1.4 Weight Update Algorithm

The conventional loss function for DeepONets is defined as

$$L \;=\; \frac{1}{N}\sum_{i=1}^{N}\big\|G_\theta(u_i)(y_i) \;-\; G(u_i)(y_i)\big\|^2,$$

where N represents the number of input functions, G_θ denotes the prediction from the DeepONet, and $G(u_i)(y_i)$ signifies the true output at the specified locations.

1. **Forward Pass:** The branch network processes the input functions, while the evaluation points are processed by the trunk network.

2. **Loss Calculation:** Determine the discrepancy between the predictions and the actual values.

3. **Backpropagation:** In DeepONets, backpropagation calculates gradients through both the Branch Net and Trunk Net, adjusting their weights simultaneously to reduce the operator prediction error using gradient descent.

4. **Parameter Update:** The weights are updated using stochastic gradient descent or its variants:

$$W_1\big(t^{n+1}\big) - W_1\big(t^n\big) = \gamma_1\,\Delta t, \quad W_2\big(t^{n+1}\big) - W_2\big(t^n\big) = \gamma_2\,\Delta t,$$

   where γ_1 and γ_2 are determined by minimizing the loss function $J(\gamma_1, \gamma_2)$.

1.5 Loss Function

The Energy-Dissipative Evolutionary DeepONet (EDE-DeepONet) utilizes a distinct loss function:

$$J(\gamma_1, \gamma_2) = \frac{1}{2} \left\| \sum_{k=1}^{p} \frac{\partial b_k(W_1^n)}{\partial W_1^n} \gamma_1 g_k(W_2^n) + \sum_{k=1}^{p} b_k(W_1^n) \frac{\partial g_k(W_2^n)}{\partial W_2^n} \gamma_2 + \frac{r^{n+1}}{\sqrt{E(u^n)}} \mathcal{N}(u^n) \right\|_2^2$$

This loss function enforces two essential characteristics:

- **Energy dissipation**: Ensures $(r^{n+1})^2 \leq (r^n)^2$, thus guaranteeing a continuous decrease in energy over time.
- **Physical consistency**: Integrates gradient-based updates from both subnetworks along with a normalized residual term associated with the system's energy $E(u^n)$.

The main elements comprise:

- **Branch/Trunk Network Gradients**: The derivatives of the branch (b_k) and trunk (g_k) outputs in relation to their respective weights (W_1, W_2).
- **Energy Term**: Incorporates the principles of thermodynamics through the residual r^{n+1} and the nonlinear operator $\mathcal{N}(u^n)$.

1.6 Discussion

In summary, the development and application of operator learning frameworks, particularly DeepONets, represents a significant advancement in the computational modeling of complex systems governed by partial differential equations. By leveraging the universal approximation capabilities of neural networks, DeepONets effectively estimate solution operators, enabling predictions that are both accurate and consistent with physical laws. The dual-branch architecture—comprising the Branch Net for input function processing and the Trunk Net for encoding evaluation points—offers adaptability and scalability for

high-dimensional problems. Furthermore, the integration of physics-informed constraints, such as energy dissipation laws, into the loss function and training process guarantees that the learned operators adhere to crucial physical principles. This holistic approach not only enhances predictive accuracy but also bridges deep learning with conventional scientific computing, laying the foundation for solving real-world engineering and scientific challenges.

Chapter 2

Review of Prior Works

2.1 Recent Works on DeepONet

Paper Name	**Authors**	**Year**	**Purpose**
Neural operator: learning maps between function spaces	N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar	2021	Introduces the DeepONet framework for learning maps between function spaces, leveraging deep neural networks for operator learning.
Neural operator: graph kernel network for partial differential equations	Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar	2020	Proposes a graph kernel network approach for solving partial differential equations using neural operators.
Fourier neural operator for parametric partial differential equations	Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar	2020	Introduces the Fourier neural operator (FNO) for solving parametric PDEs with high efficiency.
Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators	L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis	2021	Demonstrates that DeepONet can approximate nonlinear operators and solve parametric problems, leveraging the universal approximation theorem.

Table 2.1 Summary of Papers on DeepONet

2.2 Methods to Solve Partial Differential Equations

2.2.1 Separation of Variables

The method of separation of variables addresses linear homogeneous partial differential equations (PDEs) by positing that solutions take the form of products of single-variable functions, for instance, $u(x,t) = X(x)T(t)$. Inserting this into the PDE allows for the separation of variables, transforming the equation into simpler ordinary differential equations (ODEs) for individual functions. As an illustration, consider the heat equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2},$$

which results in:

$$\frac{T'}{\alpha T} = \frac{X''}{X} = -\lambda,$$

leading to the following ODEs:

$$T' + \alpha\lambda T = 0 \quad \text{and} \quad X'' + \lambda X = 0.$$

The boundary conditions provide the eigenvalues λ and eigenfunctions $X(x)$, while the superposition principle allows for the combination of solutions in more general cases.

Key features:

- Effective for scenarios with compatible geometries (e.g., rectangular domains)
- Requires linear homogeneous boundary conditions
- Commonly applied to classical PDEs such as the wave equation and Laplace's equation

Limitations:

- Limited to separable equations

- Dependent on particular coordinate systems
- Necessitates linearity and homogeneity for exact solutions

2.2.2 Wavelet-Optimized Compact Finite Difference (WOCFD) Method

The **Wavelet-optimized compact finite difference (WOCFD)** method combines wavelet analysis with high-order compact finite difference techniques to effectively address convection-diffusion equations. Notable aspects include:

- **Adaptive Grid Refinement**: Utilizes wavelet decomposition to flexibly modify grid resolution in areas with sharp gradients ($|\nabla u| \gg 0$), enhancing precision while reducing computational expenses.
- **High-Order Stencils**: Implements compact finite difference methods (for instance, 4th/6th-order) for calculating spatial derivatives, realizing minimal truncation error ($O(h^p)$) on non-uniform grids through optimized stencils:

$$\sum_{k=-N}^{N} c_k u_{i+k} = \sum_{k=-M}^{M} d_k f_{i+k}$$

- **Stability Enforcement**: Utilizes matrix eigenvalue analysis to guarantee time-stepping stability ($|\lambda_{\max}| \leq 1$) in accordance with CFL criteria.
- **Variable Coefficient Handling**: Integrates adaptive grid weighting ($w_j \propto |\mathcal{W}(u)_j|$, where $\mathcal{W}$ represents wavelet coefficients) to effectively handle mixed boundary conditions and nonlinear terms.

Tested on benchmark problems (such as Burgers' equation and Black-Scholes PDEs), the WOCFD method shows approximately a 10^2-fold decrease in grid points when compared to uniform finite differences, all while keeping L^2 errors below 10^{-5}.

2.2.3 Finite Difference Method (FDM)

The **Finite Difference Method (FDM)** solves differential equations numerically by using discrete difference formulas that are derived from Taylor series expansions to approximate derivatives. The main elements consist of:

- **Domain discretization**: This process breaks the continuous domain into a grid of points $x_i = x_0 + ih$ with a distance of h between the points.
- **Second derivative approximation**: For $u''(x_i)$:

$$u''(x_i) \approx \frac{u(x_{i+1}) - 2u(x_i) + u(x_{i-1})}{h^2} + \mathcal{O}(h^2)$$

- **Boundary conditions**: This aspect manages Dirichlet ($u(x) = g(x)$) and Neumann ($u'(x) = h(x)$) conditions by utilizing ghost points or modified stencils.
- **Algebraic systems**: This aspect translates partial differential equations like the heat equation $\partial_t u = \alpha \partial_{xx} u$ into linear systems represented as $\mathbf{Au} = \mathbf{f}$.
- **Stencil choices**: Central differences (which are second-order accurate) are preferred for points located within the interior:

$$\frac{\partial u}{\partial x} \approx \frac{u_{i+1} - u_{i-1}}{2h}$$

- **Strengths**:
  - Offers a straightforward implementation that can easily extend to higher-order approximations
  - Can be adjusted for non-uniform grids suited for complex geometries
  - Works effectively for both linear and nonlinear PDEs (such as the Navier-Stokes equations)

- **Limitations**:
  - The accuracy is influenced by the grid resolution h
  - Has difficulties dealing with irregular domains in comparison to FEM/FVM
  - Subject to stability restrictions (such as the CFL condition for problems that depend on time)

2.3 Motivation

The literature reviewed highlights a significant advancement in computational techniques for partial differential equations (PDEs): traditional methods such as separation of variables, finite difference methods, and wavelet-optimized approaches perform well for linear or structured problems, but their reliance on grid-based discretization and iterative solvers restricts scalability for nonlinear, high-dimensional, or dynamically coupled systems. DeepONet presents a revolutionary approach by utilizing operator learning to directly approximate solution maps from data, eliminating grid dependencies and facilitating real-time predictions. Its design—integrating branch-trunk networks for input-output function spaces and employing physics-informed training for embedding constraints—provides theoretical support for the universal approximation of both linear and nonlinear operators. Nonetheless, thorough validation is necessary to assess its benefits in practical applications. The forthcoming chapter will methodically assess DeepONet's performance in solving standard linear and nonlinear PDEs, comparing its accuracy, computational efficiency, and generalization to traditional methods. This examination will underscore its ability to address challenges in multiscale simulations, adaptive control systems, and real-time engineering scenarios where conventional solvers struggle.

Chapter 3

Numerical Illustrations

3.1 Heat Equation

The heat equation is a key partial differential equation that illustrates the way heat (or temperature) disperses throughout a particular area over time. This equation represents the movement of heat in a uniform and isotropic material, indicating that heat will naturally move from regions of higher temperature to those with lower temperature, gradually evening out temperature variations as time goes on. The heat equation finds extensive use in both physics and engineering for the design and examination of systems that involve heat transfer. Its mathematical characteristics, such as the maximum principle and the smoothing effect on initial discontinuities, render it a vital tool for both theoretical exploration and practical uses concerning the distribution or dissipation of energy or materials.

3.1.1 PDE, Boundary Conditions, Spatial and Temporal Domain

We begin our examination with the one-dimensional heat equation, focusing on instances where exact solutions are available. The primary partial differential equation (PDE) we are considering is:

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} \tag{3.1}$$

Utilizing the method of separation of variables with the initial condition $u(x, 0) = a \sin(\pi x)$, where a ranges from 1 to 2, and both boundaries set to zero, leads us to the analytical solution:

$$u(x, t) = a \sin(\pi x) e^{(-\pi^2 t)}.$$

3.1.2 Data Generation and Model Training

Initial training data is generated by sampling:

- **Spatial domain**: 101 equidistant points in $x \in [0, 2]$
- **Parameter range**: 3 random values of $a \in [1, 2]$ and 1 random value outside it.

For each parameter value a, we:

- Define the initial condition $u_0(x) = a \sin(\pi x)$
- Solve the time-dependent PDE using DeepONet with:
  - 4000 domain points for PDE residual evaluation
  - 400 boundary condition enforcement points
  - 50 initial condition enforcement points

The DeepONet architecture consists of:

- **Network structure**: 3 hidden layers (75 neurons/layer)
- **Activation**: Hyperbolic tangent (`tanh`)
- **Optimizer**: Adam with learning rate 10^{-3}

- **Training**: 10,000 iterations

To test generalization, we evaluate on $a \in \{1.0, 1.5, 1.8, 2.5\}$, noting that $a = 2.5$ lies outside the training distribution yet achieves comparable accuracy.

3.1.3 Plots and Error Matrix

We analyze the outcomes for various values of a, where u represents the solution derived from EDE-DeepONet, and $\hat{u}$ indicates the reference solution. We compute the mean squared error (MSE) for different values of a and show the results in Table 3.1.

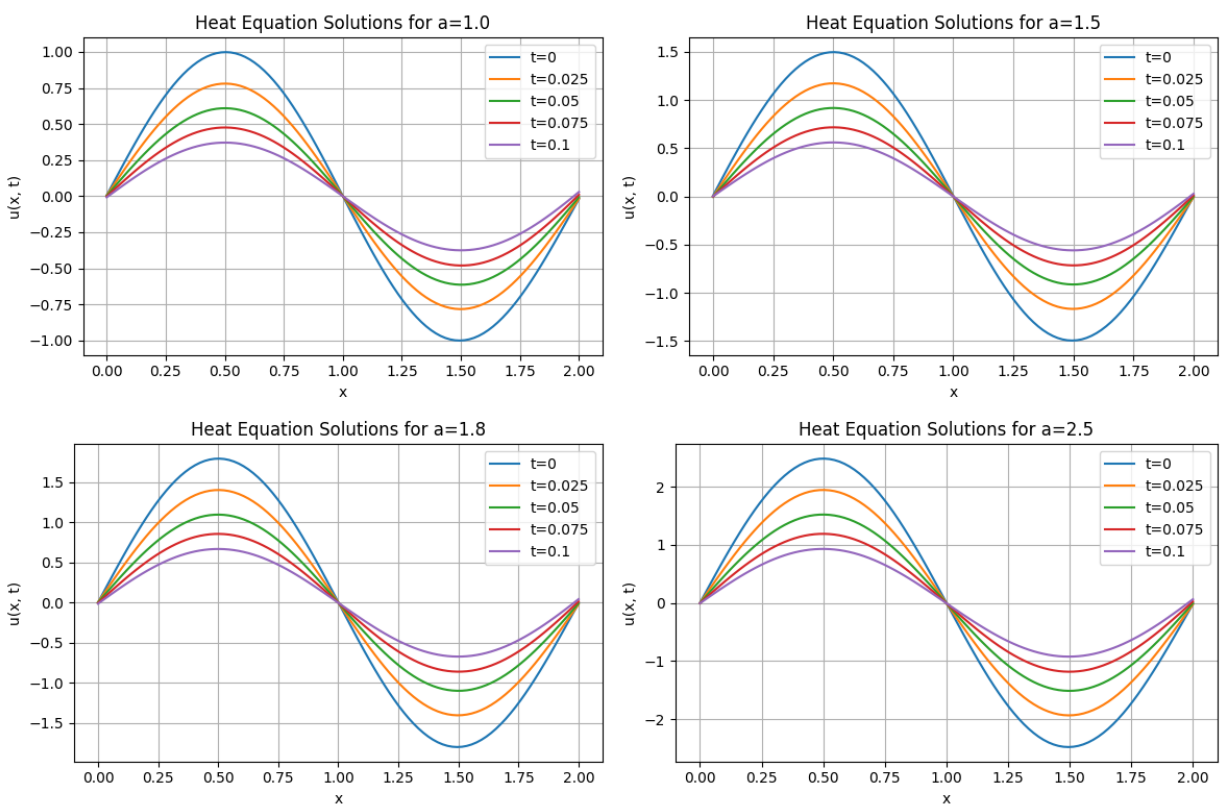

Fig. 3.1 Plots obtained at a=1.0, 1.5, 1.8 and 2.5

The performance of the model is evaluated based on the mean squared error (MSE) defined as:

$$e(u, \hat{u}) = \frac{1}{N} \sum_{k=1}^{N} (u(x_k) - \hat{u}(x_k))^2,$$

a	**T=0.0**	**T = 0.025**	**T = 0.05**	**T = 0.075**	**T = 0.1**
1.0	3.2×10^{-5}	2.2×10^{-5}	1.4×10^{-5}	7.0×10^{-6}	3.8×10^{-5}
1.5	8.9×10^{-5}	1.3×10^{-5}	8.0×10^{-6}	3.4×10^{-5}	3.4×10^{-5}
1.8	7.8×10^{-5}	2.1×10^{-5}	1.8×10^{-5}	3.1×10^{-5}	9.2×10^{-5}
2.5	2.9×10^{-5}	1.11×10^{-4}	6.3×10^{-5}	2.9×10^{-5}	24.9×10^{-5}

Table 3.1 Error Matrix - DeepONet

3.2 Parametric Heat Equation

The parametric heat equation extends the classical heat equation by incorporating a diffusivity parameter $c > 0$. This constant c represents the thermal diffusivity unique to the material, while x denotes the spatial coordinate and t stands for time. This formulation facilitates the modeling of heat diffusion across various media by adjusting c, where higher values of c correlate with quicker heat transfer. The equation is essential in contexts that require parametric analysis, such as optimizing insulation materials, simulating geothermal systems, or examining transient thermal behaviors in engineered components. Its solutions maintain important characteristics such as smoothing and maximum principles but adjust in response to c, allowing for customized predictions for heterogeneous or anisotropic materials.

3.2.1 PDE, Boundary Conditions, Spatial and Temporal Domain

We will now analyze the one-dimensional parametric heat equation, looking at scenarios where exact solutions can be found. The governing partial differential equation (PDE) is:

$$\frac{\partial u}{\partial t} = c\frac{\partial^2 u}{\partial x^2} \tag{3.2}$$

By using the method of separation of variables with the initial condition $u(x, 0) = \sin(\pi x)$, where c is within the range $[1, 2]$, we derive the analytical solution:

$$u(x, t) = \sin(\pi x)e^{(-c\pi^2 t)}.$$

3.2.2 Data Generation and Model Training

Initial training data is generated by sampling:

- **Spatial domain**: 101 equidistant points in $x \in [0, 2]$
- **Parameter range**: 3 random values of $c \in [1, 2]$ and 1 random value outside it.

For each parameter value c, we:

- Define the initial condition $u_0(x) = \sin(\pi x)$
- Solve the time-dependent PDE using DeepONet with:
  - 5000 domain points for PDE residual evaluation
  - 500 boundary condition enforcement points
  - 60 initial condition enforcement points

The DeepONet architecture consists of:

- **Network structure**: 4 hidden layers (60 neurons/layer)
- **Activation**: Hyperbolic tangent (`tanh`)
- **Optimizer**: Adam with learning rate 10^{-3}
- **Training**: 12,000 iterations

To test generalization, we evaluate on $c \in \{1.4, 1.7, 2.0, 2.3\}$, noting that $c = 2.3$ lies outside the training distribution yet achieves comparable accuracy.

3.2.3 Plots and Error Matrix

We analyze the solutions for varying values of c, where u represents the solution derived from EDE-DeepONet, and $\hat{u}$ denotes the reference solution. We compute the Mean Squared Error (MSE) for different values of c and showcase the findings in Table 3.2.

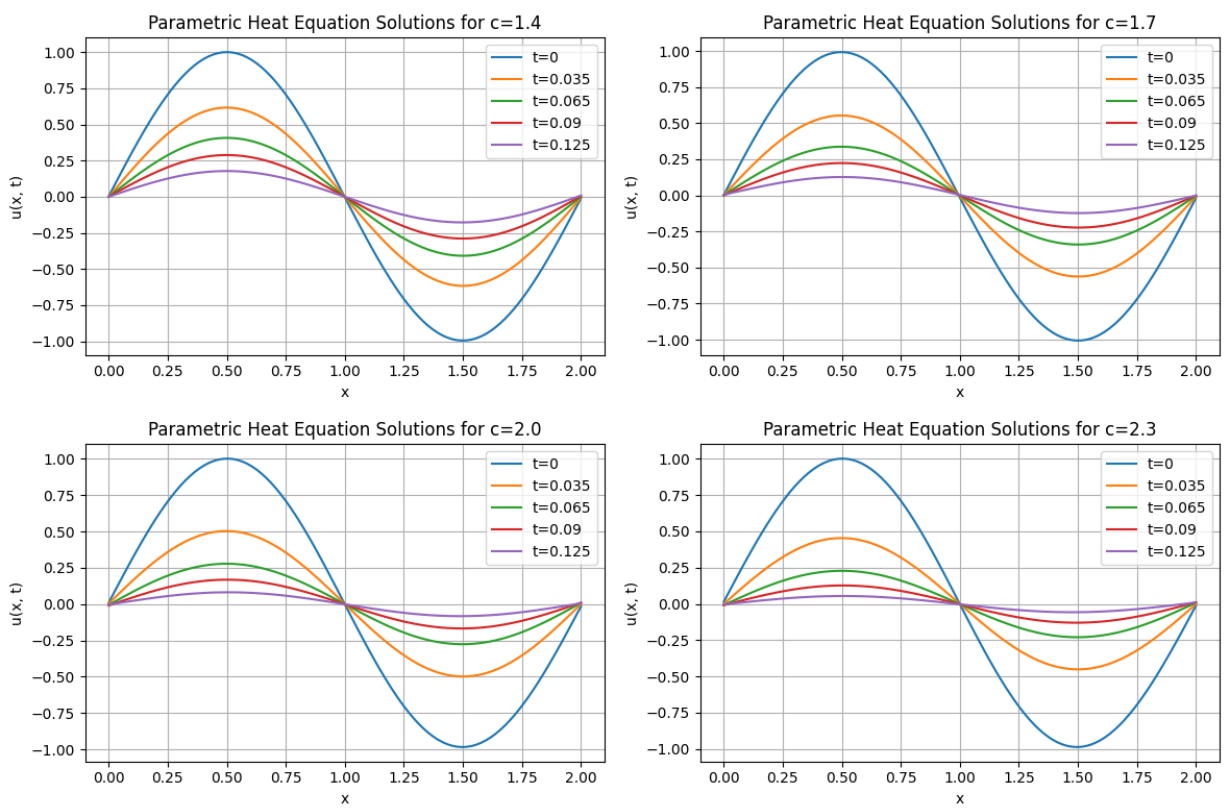

Fig. 3.2 Plots obtained at c=1.4, 1.7, 2.0 and 2.3

The performance of the model is evaluated based on the mean squared error (MSE) defined as:

$$e(u, \hat{u}) = \frac{1}{N} \sum_{k=1}^{N} (u(x_k) - \hat{u}(x_k))^2,$$

c	**T=0.0**	**T = 0.035**	**T = 0.065**	**T = 0.09**	**T = 0.125**
1.4	1.1×10^{-5}	0.4×10^{-5}	0.1×10^{-5}	0.1×10^{-5}	0.4×10^{-5}
1.7	11.4×10^{-5}	3.4×10^{-5}	1.0×10^{-5}	0.4×10^{-5}	0.9×10^{-5}
2.0	5.1×10^{-5}	0.4×10^{-5}	0.5×10^{-5}	1.6×10^{-5}	1.1×10^{-5}
2.3	5.3×10^{-5}	0.2×10^{-5}	0.8×10^{-5}	1.8×10^{-5}	1.9×10^{-5}

Table 3.2 Error Matrix - DeepONet

3.3 2D Convection Diffusion Equation

The equation is a key partial differential equation that represents the combined impacts of convection (movement due to the bulk motion of fluid) and diffusion (spreading caused by random molecular movement) on a physical quantity such as temperature, concentration, or pollutants within a medium. Its general expression is

$$\frac{\partial c}{\partial t} + \vec{v} \cdot \nabla c = D \nabla^2 c + R,$$

where $c(x, y, t)$ denotes the variable of interest (like concentration or temperature), $\vec{v}$ symbolizes the velocity field that reflects convection, D represents the diffusion coefficient, and R encapsulates sources or sinks (generation or loss) of c. In two-dimensional scenarios, this equation illustrates how substances or heat are transported by flow while also dispersing due to diffusion in both the x and y directions, making it crucial for modeling phenomena in environmental engineering, chemical processes, atmospheric studies, and biological systems. The interaction between convection and diffusion results in complex behaviors such as the establishment of gradients, fronts, or mixing regions, and the equation is frequently solved using numerical methods for practical geometries and flow conditions. Its adapt-

ability also permits extensions to systems with changing coefficients, nonlinear reactions, or interactions between multiple species, solidifying its status as a foundational element of mathematical modeling in transport processes.

3.3.1 PDE, Boundary Conditions, Spatial and Temporal Domain

The governing partial differential equation (PDE) is expressed as:

$$\frac{\partial u}{\partial t} = 0.5\frac{\partial^2 u}{\partial x^2} + 0.5\frac{\partial^2 u}{\partial y^2} - 5\frac{\partial u}{\partial x} - 5\frac{\partial u}{\partial y} \tag{3.3}$$

Applying the separation of variables method to the initial condition $u(x, y, 0) = f(x, y) = \sin(\pi x)\sin(\pi y)$, we derive the eigenfunctions $X(x)$ and $Y(y)$ resulting in

$$X(x)Y(y) = e^{5x}e^{5y}\sin(m\pi x)\sin(n\pi y)$$

for $m, n = 1, 2, \ldots$. This leads to zero boundary conditions and yields the analytical solution:

$$u(x, y, t) = \sum_{m=1}^{\infty}\sum_{n=1}^{\infty} A_{m,n}\, e^{\left(-0.5m^2\pi^2 - 0.5n^2\pi^2 - 25\right)t}\, e^{5(x+y)}\sin(m\pi x)\sin(n\pi y)$$

where

$$A_{m,n} = 4\int_0^1\int_0^1 e^{-5(x+y)}\sin(\pi x)\sin(\pi y)\sin(m\pi x)\sin(n\pi y)\,dx\,dy$$

3.3.2 Data Generation and Model Training

Initial training data is generated by sampling:

- **Spatial domain**: 101 x 101 equidistant points in $x \in [0, 1]$ and $y \in [0, 1]$
- **Parameter range**: Diffusion Coefficient $d = 0.5$, Convection Coefficient $c = 5$
- Define the initial condition $u(x, y, 0) = f(x, y) = \sin(\pi x)\sin(\pi y)$

- Solve the time-dependent PDE using DeepONet with:
  - 8000 domain points for PDE residual evaluation
  - 600 boundary condition enforcement points
  - 40 initial condition enforcement points

The DeepONet architecture consists of:

- **Network structure**: 4 hidden layers (75 neurons/layer), 3 hidden layers (150 neurons/layer), 2 hidden layers (225 neurons/layer)
- **Activation**: Hyperbolic tangent (`tanh`)
- **Optimizer**: Adam with learning rate 10^{-4}
- **Training**: 20,000 iterations

To test generalization, we evaluate on time steps, $T \in \{0.0, 0.05, 0.09, 0.12\}$.

3.3.3 Plots and Error Matrix

We compare the solutions for different values of T where u is the solution obtained by EDE-DeepONet, and $\hat{u}$ is the reference solution. We calculate the different error metrics for different values of T and present the results in Table 3.3.

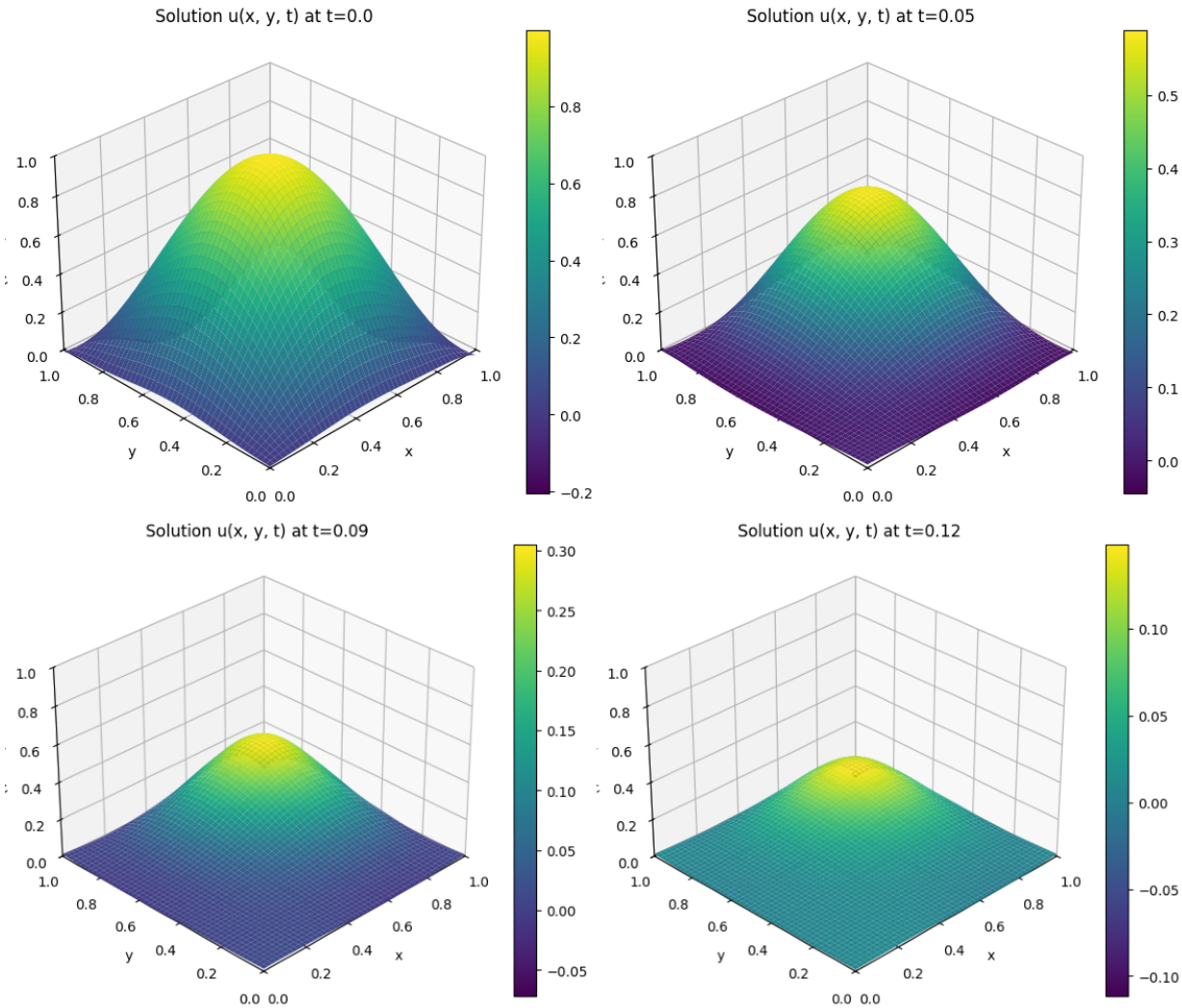

Fig. 3.3 Plots obtained at T=0.0, 0.05, 0.09 and 0.12

We also visualize the error plots for all points in the domain.

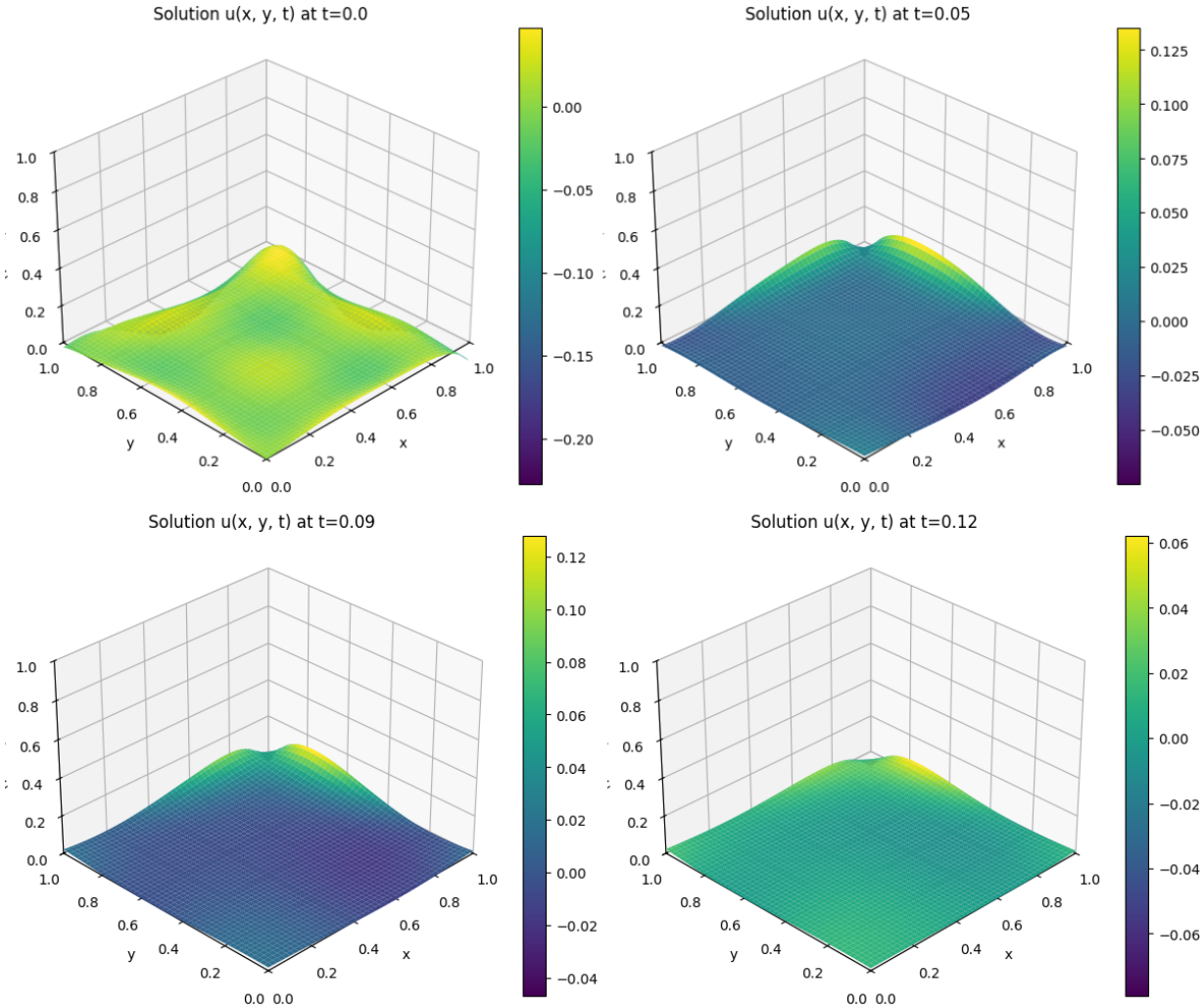

Fig. 3.4 Plots obtained at T=0.0, 0.05, 0.09 and 0.12

The performance of the model is evaluated based on 3 metrics defined as:

1) Mean Squared Error:

$$e(u, \hat{u}) = \frac{1}{N} \sum_{k=1}^{N} (u(x_k) - \hat{u}(x_k))^2,$$

2) L2 Norm:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} (u(x_i) - \hat{u}(x_i))^2}$$

3) L-infinity Norm:

$$\|\mathbf{x}\|_\infty = \max_{1\leq i\leq n} |(u(x_i) - \hat{u}(x_i))|$$

	t = 0.00	t = 0.05	t = 0.09	t = 0.12
MSE	0.001025	0.000255	0.000231	0.000059
L2 norm	3.201286	1.596813	1.520175	0.768905
L(infinity) norm	0.207155	0.097484	0.092527	0.104439

Table 3.3 Error metrics at different time stamps.

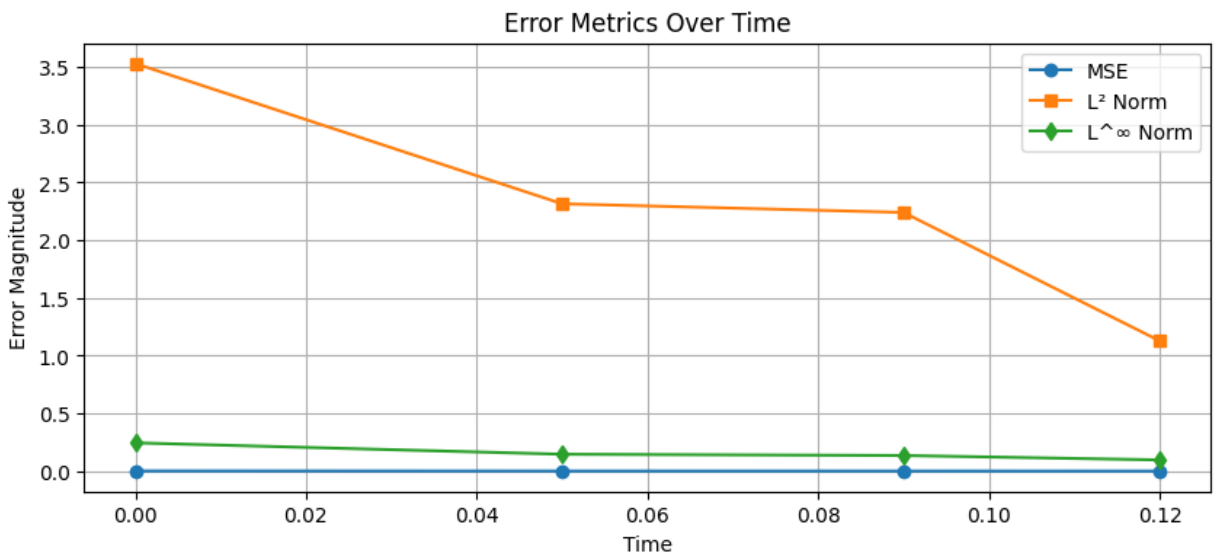

Fig. 3.5 Error Metrics overtime

3.4 Analysis of Illustrations

The use of DeepONet for canonical PDEs has produced encouraging yet complex outcomes: it achieved a high level of precision (with an MSE on the order of 0.00001) for both the basic and parametric heat equations, indicating effective operator learning in one-dimensional linear contexts. Nevertheless, its performance declined for the two-dimensional convection-diffusion issue, exhibiting localized divergence near adiabatic boundaries and in areas with steep gradient transitions. This indicates that although DeepONet performs well in lower-dimensional or smoothly changing systems, difficulties emerge in higher-dimensional spaces

featuring mixed boundary conditions, likely due to inadequate boundary layer dynamics resolution or limited training data in crucial areas. These results lead to a targeted exploration into architectural improvements—such as combining Fourier techniques with DeepONet or adaptive sampling of the trunk network—to enhance stability for multiscale two- and three-dimensional challenges. Furthermore, incorporating domain decomposition techniques or weights that account for boundary loss could mitigate spatial instability, creating a link between theoretical operator approximation and its use in real-world engineering.

Chapter 4

Conclusion & Discussion

DeepONet provides several significant benefits as a neural operator for addressing partial differential equations (PDEs). It displays an outstanding capability for approximating non-linear PDEs and their derivatives, making it very effective for a variety of operator learning tasks. Its modular and adaptable design facilitates straightforward adjustments for different operator problems, and utilizing pretraining methods can greatly decrease the amount of data needed for successful learning. In some cases, DeepONet is also computationally efficient, allowing for quick inference after training. Moreover, it is supported by a robust theoretical framework, featuring proven error bounds and generalization estimates that give assurance in its predictive accuracy.

Despite its advantages, DeepONet has its drawbacks. It struggles with problems that feature discontinuous solutions, which can hinder its ability to approximate effectively. The effectiveness of DeepONet largely depends on the design of the network and the tuning of hyperparameters, necessitating careful adjustments to achieve the best performance. Although pretraining can enhance data efficiency, certain challenging issues still require a significant amount of training data. The training phase can also be demanding in terms of resources, particularly for intricate or high-dimensional problems. Lastly, while theoretical findings offer useful perspectives, some inquiries about the limitations and assurances of

DeepONet remain unanswered, indicating a need for further investigation.

4.1 Conclusion

DeepONet has demonstrated its capability as a versatile and powerful framework for learning and forecasting solutions to complex partial differential equations, especially in the realm of parametric and multi-dimensional challenges like the heat and convection-diffusion equations. By adeptly representing both the input function and the domain of evaluation, DeepONet exhibits strong generalization across diverse initial and boundary conditions, even for parameters that lie outside the training range. Its solid performance, efficiency in computation, and capacity to integrate physical constraints make it a crucial resource for speeding up scientific exploration and engineering design.

4.2 Future Work

Potential future research could aim to enhance DeepONet's capability to tackle more complex situations, such as the Black Scholes Model for Down Out Barrier Call options. Integrating sophisticated methods like uncertainty quantification, adaptive sampling, and multi-fidelity data combination could significantly boost the model's dependability and practical use in real-world scenarios. Furthermore, investigating hybrid structures that merge neural operators with conventional numerical methods might lead to better scalability and precision for high-dimensional or large-scale industrial challenges. These developments would expand the influence of DeepONet in the fields of scientific computing and engineering.